



МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ – МСХА  
имени К. А. ТИМИРЯЗЕВА»  
(ФГБОУ ВО РГАУ – МСХА имени К. А. Тимирязева)

Институт экономики и управления  
Кафедра статистики и кибернетики  
Направление: 09.03.02 Информационные системы и технологии  
Направленность: Большие данные и машинное обучение  
Методы и средства проектирования информационных систем и технологий

## КУРСОВОЙ ПРОЕКТ

на тему: «Проектирование и разработка веб-приложения «Цифровой агроном» с  
ИИ-ассистентом для поддержки принятия решений в растениеводстве».

Выполнил:

Студент 3 курса группы Д-Э-311

Бондарев И. А.

Дата регистрации КП

на кафедре: \_\_\_\_\_

Допущен к защите

Руководитель:

\_\_\_\_\_

учёная степень, учёное звание, ФИО

Члены комиссии:

\_\_\_\_\_

|                                    |         |
|------------------------------------|---------|
| учёная степень, учёное звание, ФИО | подпись |
|------------------------------------|---------|

\_\_\_\_\_

|                                    |         |
|------------------------------------|---------|
| учёная степень, учёное звание, ФИО | подпись |
|------------------------------------|---------|

\_\_\_\_\_

|                                    |         |
|------------------------------------|---------|
| учёная степень, учёное звание, ФИО | подпись |
|------------------------------------|---------|

Оценка \_\_\_\_\_

Дата защиты \_\_\_\_\_

Москва 2026

# Содержание

|                                                                                                        |           |
|--------------------------------------------------------------------------------------------------------|-----------|
| <b>Введение</b>                                                                                        | <b>3</b>  |
| <b>1. Глава 1. Теоретическая часть</b>                                                                 | <b>5</b>  |
| 1.1. Эволюция и современное состояние систем поддержки принятия решений в сельском хозяйстве . . . . . | 5         |
| 1.2. Методы искусственного интеллекта и машинного обучения в растениеводстве . . . . .                 | 6         |
| 1.3. Проблемы импортозамещения и доступности технологий в РФ .                                         | 6         |
| 1.4. Технологический стек для разработки веб-приложения . . . . .                                      | 7         |
| 1.5. Критерии качества и метрики оценки системы . . . . .                                              | 8         |
| <b>2. Глава 2. Практическая часть</b>                                                                  | <b>10</b> |
| 2.1. Архитектура программного комплекса . . . . .                                                      | 10        |
| 2.2. Реализация серверной части . . . . .                                                              | 10        |
| 2.3. Реализация клиентской части и описание функций . . . . .                                          | 14        |
| 2.4. Логирование и запуск системы . . . . .                                                            | 15        |
| 2.5. Результаты тестирования . . . . .                                                                 | 15        |
| <b>Заключение</b>                                                                                      | <b>16</b> |
| <b>Список использованных источников</b>                                                                | <b>17</b> |
| <b>Приложение</b>                                                                                      | <b>19</b> |
| Листинг Б.1 — <code>app.py</code> . . . . .                                                            | 23        |
| Листинг Б.2 — <code>crop_comparator.py</code> . . . . .                                                | 26        |
| Листинг Б.3 — <code>rotation_predictor.py</code> . . . . .                                             | 28        |
| Листинг Б.4 — <code>ai_assistant.py</code> . . . . .                                                   | 30        |
| Листинг Б.5 — <code>ml_analytics.py</code> . . . . .                                                   | 32        |
| Листинг Б.6 — <code>config.py</code> . . . . .                                                         | 34        |
| Листинг Б.7 — <code>index.html</code> . . . . .                                                        | 35        |
| Листинг Б.8 — <code>chat.html</code> . . . . .                                                         | 38        |

# Введение

В современной цифровой среде актуальность интеллектуальных систем поддержки принятия решений в сельском хозяйстве существенно возрастает. Растениеводство сталкивается с рядом вызовов: изменение климата, распространение новых болезней, необходимость оптимизации ресурсов. Традиционные методы консультации не всегда доступны фермерам в удалённых регионах.

Цифровая трансформация сельского хозяйства (AgriTech) предлагает решения на базе искусственного интеллекта. Зарубежные продукты (Plantix, Climate FieldView) [1][2] демонстрируют возможность автоматической диагностики болезней и прогнозирования урожайности. Однако многие из них недоступны в РФ, требуют платной подписки или не учитывают российскую специфику.

В условиях санкционного давления возникли проблемы с доступом к зарубежной инфраструктуре разработки (PyPI.org, GitHub, Hugging Face) [4], что потребовало поиска альтернативных путей: использование зеркал, ручную установку совместимых версий, отказ от проблемных библиотек.

Целью данной работы является проектирование, разработка и экспериментальная апробация веб-приложения «Цифровой агроном» с ИИ-ассистентом, обеспечивающего прогнозирование урожайности на основе ML-модели и консультационную поддержку.

Для достижения цели требуется решить следующие задачи:

- 1) Провести анализ систем поддержки принятия решений в сельском хозяйстве.
- 2) Изучить проблемы импортозамещения и доступности технологий в РФ.
- 3) Выбрать оптимальный стек технологий.
- 4) Спроектировать модульную архитектуру веб-приложения.
- 5) Реализовать программный прототип.
- 6) Провести тестирование и оценить качество работы системы.

Объектом исследования являются процессы поддержки принятия решений в растениеводстве. Предметом исследования выступают методы машинного обучения для прогнозирования урожайности и технологии построения ИИ-ассистентов.

Практическая значимость работы заключается в возможности использования разработанного приложения для помощи фермерским хозяйствам.

Структура работы включает введение, две главы (теоретическую и практическую), заключение, библиографический список и приложение.

# 1. Глава 1. Теоретическая часть

## 1.1. Эволюция и современное состояние систем поддержки принятия решений в сельском хозяйстве

История развития систем поддержки принятия решений (СППР) в сельском хозяйстве отражает общую эволюцию информационных технологий — от простых статистических отчётов до сложных интеллектуальных систем [13].

На начальном этапе (1960–1980-е годы) использовались бумажные агрономические карты и справочники. С появлением персональных компьютеров начали разрабатываться первые электронные базы данных по сортам культур, удобрениям и средствам защиты растений.

В 1990-е годы с развитием спутниковых технологий и GPS возникло понятие «точного земледелия» (precision agriculture) [12]. Появились системы мониторинга полей, позволяющие учитывать пространственную неоднородность почв и вегетационный индекс растений (NDVI).

Современный этап (2010-е годы – настоящее время) характеризуется массовым внедрением технологий искусственного интеллекта, машинного обучения и компьютерного зрения. Это стало возможным благодаря росту вычислительных мощностей, доступности больших размеченных датасетов (PlantVillage, 54 000+ изображений) и развитию библиотек глубокого обучения.

На сегодняшний день существуют следующие категории систем: диагностика по фото (Plantix) [1], мониторинг полей (Climate FieldView) [2], экспертные системы (Agrobase) [3], ERP для АПК, ИИ-ассистенты.

Анализ существующих решений показывает, что большинство зарубежных продуктов недоступны в РФ или требуют платной подписки, экспертные системы часто содержат устаревшую информацию, а универсальные ИИ-ассистенты не обладают специализированными знаниями в агрономии.

## 1.2. Методы искусственного интеллекта и машинного обучения в растениеводстве

Современные методы искусственного интеллекта открывают новые возможности для автоматизации диагностики, прогнозирования и консультирования в сельском хозяйстве.

Компьютерное зрение. Диагностика болезней по фотографиям листьев является задачей классификации изображений. Традиционные методы основаны на выделении признаков вручную (цветовые гистограммы, текстурные дескрипторы) с последующей классификацией (SVM, Random Forest). Методы глубокого обучения используют свёрточные нейронные сети (CNN), которые автоматически извлекают признаки. Наиболее популярные архитектуры: ResNet50 (99,2%), InceptionV3 (98,5%), YOLOv8 (97,7%) [15].

Прогнозирование урожайности. Это задача регрессии — предсказания непрерывного значения на основе набора признаков: климатических (температура, осадки), почвенных (тип почвы, pH), агротехнических (культура, удобрения) и временных (год, сезон). Наиболее эффективные модели: Random Forest [10] (устойчив к переобучению), Gradient Boosting (высокая точность), LSTM [14] (учёт временных зависимостей).

Большие языковые модели (LLM). LLM — нейросетевые архитектуры на основе трансформеров с механизмом внимания (attention) [11], обученные на огромных объёмах текстовых данных. Применение в сельском хозяйстве: консультации по уходу, диагностика по описанию, рекомендации по лечению, интерпретация ML-прогнозов. Архитектура RAG (Retrieval-Augmented Generation) дополняет LLM поиском по внешней базе знаний.

## 1.3. Проблемы импортозамещения и доступности технологий в РФ

С 2022 года российские разработчики столкнулись с ограничениями доступа к зарубежной инфраструктуре [4].

Блокировка ресурсов. Затруднена работа PyPI.org (основной репозиторий Python), GitHub (хостинг кода) и Hugging Face (платформа для моделей ML). Блокировка домена .org сделала невозможной автоматическую установку библиотек через `pip install`.

Проблемы совместимости. Выход NumPy 2.0 в 2024 году создал несовместимость с библиотеками (OpenCV, Ultralytics), собранными с использованием NumPy 1.x API [5].

Найденные решения: использование зеркал PyPI, ручная установка совместимых версий (`'numpy==1.24.3'`), отказ от проблемных библиотек, использование OpenRouter API вместо локального запуска LLM, замена requests на встроенную urllib.

## 1.4. Технологический стек для разработки веб-приложения

На основе проведённого анализа и с учётом выявленных ограничений был выбран следующий технологический стек.

Для реализации бэкенда (серверной части) используется веб-фреймворк Flask версии 3.0 [6]. Данный фреймворк является лёгким, минималистичным и предоставляет все необходимые возможности для создания веб-приложения с REST API. Язык программирования — Python версии 3.13, который обладает обширной экосистемой библиотек для машинного обучения и научных вычислений, что критически важно для реализации ML-модели.

Для машинного обучения выбраны библиотеки scikit-learn 1.3.2 [7], pandas 2.0.3 и numpy 1.24.3. Эти версии являются стабильными и совместимыми друг с другом, что особенно важно в условиях проблем с установкой более новых версий, вызванных выходом NumPy 2.0 [5]. В качестве HTTP-клиента используется встроенная библиотека urllib, что позволило отказаться от внешней зависимости requests и снизить количество потенциальных проблем при установке.

Для реализации ИИ-ассистента выбран провайдер OpenRouter API [8], который агрегирует множество языковых моделей и стабильно работает на территории Российской Федерации. Используется модель GPT-OSS-120B — бесплатная, стабильная, с хорошим качеством ответов на русском языке. Для обеспечения отказоустойчивости реализован fallback-режим на основе локальных правил (эвристик), который активируется при недоступности API и обеспечивает базовую функциональность.

Фронтенд (клиентская часть) реализован с использованием стандартных веб-технологий: HTML5 для разметки страниц, CSS3 для стилизации и

обеспечения адаптивного дизайна (с использованием медиазапросов), а также JavaScript (ES6) для асинхронного взаимодействия с сервером через fetch API.

В качестве источника данных используется открытый датасет Продовольственной и сельскохозяйственной организации ООН (FAO) [9], содержащий 28 242 записи об урожайности различных культур в разных странах за период с 1990 по 2013 год. Хранение данных осуществляется в формате CSV, что достаточно для статического датасета и не требует развёртывания полноценной системы управления базами данных.

## 1.5. Критерии качества и метрики оценки системы

Для объективной оценки разработанной системы были определены следующие категории метрик: метрики ML-модели прогнозирования, технические метрики и бизнес-метрики.

Метрики ML-модели позволяют оценить точность прогнозирования урожайности. Коэффициент детерминации  $R^2$  показывает, какую долю дисперсии целевой переменной объясняет модель. Значение 1 соответствует идеальному предсказанию. Среднеквадратичная ошибка (RMSE) измеряет величину ошибки прогноза в единицах целевой переменной (ц/га). Средняя абсолютная ошибка (MAE) аналогична RMSE, но менее чувствительна к выбросам. Относительная точность вычисляется как  $1 - \text{MAE}/\bar{y}$  и показывает процентную точность прогноза относительно среднего значения урожайности.

Технические метрики характеризуют производительность системы. Время инференса ML-модели — это время от получения входных параметров до выдачи прогноза, целевое значение составляет менее 100 миллисекунд. Время ответа API чата включает отправку запроса к OpenRouter API [8] и получение ответа, целевое значение — менее 5 секунд (с учётом сетевой задержки). Время загрузки страницы измеряется инструментами разработчика браузера и не должно превышать 2 секунд.

Бизнес-метрики оценивают практическую полезность системы. Процент автоответов чата показывает, какая доля пользовательских запросов может быть обработана без привлечения API (с использованием fallback-режима), целевое значение — более 65%. Уверенность прогноза определяется



вероятностью, которую выдаёт ML-модель, и должна составлять более 70%.  
Покрытие культур — количество различных культур, для которых система может сделать прогноз, целевое значение — более 5 культур.

## 2. Глава 2. Практическая часть

### 2.1. Архитектура программного комплекса

Разработанное веб-приложение «Цифровой агроном» построено по классической клиент-серверной архитектуре. Система состоит из следующих основных модулей: модуль веб-интерфейса (Frontend), серверный модуль (Backend) на Flask [6], модуль прогнозирования урожайности (yield\_predictor.py), модуль прогнозирования на будущие годы (trend\_forecaster.py), модуль сравнения культур (crop\_comparator.py), модуль севооборота (rotation\_predictor.py), модуль ИИ-ассистента (ai\_assistant.py) [8] и модуль агрономической аналитики (ml\_analytics.py). Все ML-модели обучены на реальных данных FAO [9] (28 242 записи).

### 2.2. Реализация серверной части

Серверная часть реализована на языке Python с использованием микрофреймворка Flask [6]. Основной файл приложения — app.py — содержит маршруты для обработки запросов. Проект имеет следующую организацию:

```
digital-agronomist/
├── app.py                # Главный сервер
├── config.py             # Загрузка переменных окружения
├── requirements.txt      # Зависимости
├── .env                 # API ключи
├── services/
│   ├── ai_assistant.py  # ИИ-ассистент
│   └── ml_analytics.py  # Обёртка для ML-моделей
├── models/
│   ├── yield_predictor.py # ML прогноз урожайности
│   ├── trend_forecaster.py # Прогноз на будущие годы
│   ├── crop_comparator.py # ML сравнение культур
│   └── rotation_predictor.py # Севооборот
└── templates/
    ├── index.html       # Главная страница
    └── chat.html         # Страница чата
```

```
data/  
    crop_yield.csv                # Датасет FAO \cite{faostat}
```

**Основной сервер.** Ниже представлен листинг с основными маршрутами файла app.py.

```
from flask import Flask, render_template, request, jsonify  
from services.ml_analytics import ml_analytics  
from services.ai_assistant import ai_assistant
```

```
app = Flask(__name__)
```

```
@app.route('/')  
def index():  
    return render_template('index.html')
```

```
@app.route('/chat')  
def chat():  
    return render_template('chat.html')
```

```
@app.route('/api/predict-yield', methods=['POST'])  
def predict_yield():  
    data = request.get_json()  
    result = ml_analytics.predict_yield(  
        crop=data.get('crop'),  
        area=data.get('area'),  
        year=int(data.get('year')),  
        temperature=float(data.get('temperature')),  
        rainfall=float(data.get('rainfall')),  
        pesticides=float(data.get('pesticides'))  
    )  
    return jsonify(result)
```

```
@app.route('/api/forecast', methods=['POST'])
```

```

def forecast():
    data = request.get_json()
    result = ml_analytics.forecast_future(
        crop=data.get('crop'),
        start_year=int(data.get('start_year')),
        end_year=int(data.get('end_year')),
        temp_trend=float(data.get('temp_trend'))
    )
    return jsonify(result)

@app.route('/api/compare-crops', methods=['POST'])
def compare_crops():
    data = request.get_json()
    result = ml_analytics.compare_crops(
        temperature=float(data.get('temperature')),
        rainfall=float(data.get('rainfall')),
        pesticides=float(data.get('pesticides')),
        year=int(data.get('year'))
    )
    return jsonify(result)

@app.route('/api/rotation-advice', methods=['POST'])
def rotation_advice():
    data = request.get_json()
    result = ml_analytics.rotation_advice(data.get('last_crop'))
    return jsonify(result)

@app.route('/api/chat', methods=['POST'])
def chat_api():
    data = request.get_json()
    result = ai_assistant.chat(data.get('message'))
    return jsonify({'response': result['response']})

```

**ML-модель прогнозирования урожайности.** Листинг модуля `yield_predictor.py` представлен ниже. Модель построена на основе алгоритма Random

Forest [10].

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler, LabelEncoder

class YieldPredictor:
    def __init__(self):
        self.model = RandomForestRegressor(n_estimators=200, random_st

        self.scaler = StandardScaler()
        self.crop_encoder = LabelEncoder()
        self.area_encoder = LabelEncoder()

    def predict(self, crop, area, year, temperature, rainfall, pestici
        crop_enc = self.crop_encoder.transform([crop])[0]
        area_enc = self.area_encoder.transform([area])[0]
        features = [[crop_enc, area_enc, year, temperature, rainfall,
        features_scaled = self.scaler.transform(features)
        return self.model.predict(features_scaled)[0]
```

**ИИ-ассистент.** Листинг модуля ai\_assistant.py представлен ниже. Для взаимодействия с LLM используется OpenRouter API [8].

```
import urllib.request
import json

class AIAssistant:
    def __init__(self):
        self.api_key = os.getenv('OPENROUTER_API_KEY')
        self.model = 'gpt-oss-120b:free'
        self.system_prompt = "Ты - опытный агроном, ИИ-ассистент для ф

    def chat(self, message):
        response = self._send_request(message)
        if response:
            return response
```

```

        return self._fallback_response(message)

def _send_request(self, message):
    payload = {
        "model": self.model,
        "messages": [{"role": "user", "content": message}]
    }
    req = urllib.request.Request(
        self.api_url,
        data=json.dumps(payload).encode(),
        headers={"Authorization": f"Bearer {self.api_key}"})
    with urllib.request.urlopen(req) as response:
        return json.loads(response.read())['choices'][0]['message']

```

## 2.3. Реализация клиентской части и описание функций

Клиентская часть реализована в виде двух HTML-страниц: index.html (главная страница) и chat.html (страница чата). На главной странице расположены четыре основные функциональные карточки.

Функция 1: ML прогноз урожайности. Пользователь выбирает культуру, регион, задаёт год, температуру, осадки и пестициды. После нажатия кнопки «Рассчитать прогноз» отправляется POST-запрос к эндпоинту /api/predict-yield. Результат включает числовое значение урожайности, рекомендацию и указание модели (внешний вид результата представлен на рисунке 1 в приложении).

Функция 2: ML прогноз на будущие годы. Пользователь задаёт культуру, период прогноза и тренд температуры. Результат отображается в виде таблицы с прогнозами по годам (внешний вид результата представлен на рисунке 2 в приложении).

Функция 3: ML сравнение культур. Пользователь задаёт климатические условия. Система определяет рейтинг культур с указанием вероятности для каждой культуры (внешний вид результата представлен на рисунке 3 в приложении).

Функция 4: ML севооборот. Пользователь вводит культуру, которая рос-

ла в прошлом сезоне. Система предоставляет рекомендации по чередованию культур (внешний вид результата представлен на рисунке 4 в приложении).

Страница чата с ИИ-ассистентом позволяет задавать вопросы в текстовом поле и получать ответы от большой языковой модели (внешний вид страницы представлен на рисунке 5 в приложении). Пример ответа ассистента на вопрос «как копать картошку» представлен на рисунке 6 в приложении.

## **2.4. Логирование и запуск системы**

Для мониторинга работы приложения в консоль сервера выводятся информационные сообщения. Процесс успешного запуска сервера представлен на рисунке 7 в приложении, логи работы чат-бота — на рисунке 8 в приложении.

## **2.5. Результаты тестирования**

В ходе тестирования были получены следующие результаты. Модель Random Forest [10] показала высокую точность на тестовой выборке:  $R^2 = 0,985$ , относительная точность 93,5%. Для культуры Cassava, региона Albania, года 2025, температуры 24°C, осадков 150 мм и пестицидов 100 т прогнозируемая урожайность составила 98 161,5 ц/га. Для культуры Cassava на период 2025–2030 годы при тренде температуры 0,1°C в год прогнозируется рост урожайности на 10,6%. При климатических условиях 24°C, 150 мм осадков, 100 т пестицидов, год 2025 лучшей культурой признана Wheat с вероятностью 24,7%. Для прошлой культуры potatoes рекомендуются культуры Legumes, Cabbage и Onion. ИИ-ассистент [8] при запросе «как копать картошку» предоставил структурированные рекомендации за время менее 3 секунд.

## Заключение

В ходе выполнения курсовой работы была достигнута поставленная цель — разработано веб-приложение «Цифровой агроном» с ИИ-ассистентом для поддержки принятия решений в растениеводстве.

Проведён анализ существующих систем поддержки принятия решений в сельском хозяйстве [1][2][3], выявлены их достоинства и недостатки, обоснована актуальность разработки отечественной системы. Исследованы проблемы импортозамещения и доступности технологий на территории РФ [4] (блокировка PyPI, GitHub, Hugging Face, несовместимость NumPy 1.x/2.x [5]), найдены обходные пути: использование зеркал PyPI, фиксация версий библиотек, отказ от проблемных зависимостей.

Выбран оптимальный технологический стек: Python, Flask [6], scikit-learn [7], pandas, OpenRouter API [8], HTML, CSS, JavaScript. Спроектирована модульная архитектура, включающая модули ML-прогнозирования, ИИ-ассистента и веб-интерфейса.

Реализован программный прототип. Разработана ML-модель прогнозирования урожайности на основе Random Forest [10], обученная на реальных данных FAO [9] (28 242 записи). Качество модели составило  $R^2 = 0,985$  на тестовой выборке при относительной точности 93,5%. Реализованы модули прогноза на будущие годы, сравнения культур и севооборота. Создан ИИ-ассистент на основе OpenRouter API [8] (модель GPT-OSS-120B) с fallback-режимом для обеспечения отказоустойчивости. Разработан адаптивный веб-интерфейс.

Проведённое тестирование подтвердило работоспособность всех модулей: время инференса ML-модели составило 47 мс, время ответа ИИ-ассистента — менее 3 секунд. Система успешно обрабатывает запросы на прогнозирование, сравнение культур, севооборот и консультации.

Разработанное веб-приложение может использоваться фермерскими хозяйствами и агрономами для поддержки принятия решений в растениеводстве. Перспективы развития включают добавление компьютерного зрения [15], интеграцию с погодными API и расширение базы знаний.



## Список использованных источников

1. Plantix — диагностика болезней растений по фото. URL: <https://plantix.net> (дата обращения: 01.06.2026).
2. Climate FieldView — платформа точного земледелия. URL: <https://climate.com> (дата обращения: 01.06.2026).
3. Agrobase — экспертная система по средствам защиты растений. URL: <https://agrobase.ru> (дата обращения: 01.06.2026).
4. Ограничения доступа к зарубежным IT-ресурсам на территории РФ. Аналитический обзор. — М.: Институт развития интернета, 2024.
5. NumPy 2.0 Release Notes. URL: <https://numpy.org/doc/stable/release/2.0.0-notes.html> (дата обращения: 01.06.2026).
6. Flask Documentation. URL: <https://flask.palletsprojects.com> (дата обращения: 01.06.2026).
7. Pedregosa F. et al. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825–2830.
8. OpenRouter API Documentation. URL: <https://openrouter.ai/docs> (дата обращения: 01.06.2026).
9. FAO Statistical Database. URL: <https://www.fao.org/faostat> (дата обращения: 01.06.2026).
10. Breiman L. Random Forests // Machine Learning. — 2001. — Vol. 45, № 1. — P. 5–32.
11. Vaswani A. et al. Attention Is All You Need // Advances in Neural Information Processing Systems. — 2017. — P. 5998–6008.
12. Системы бесконтактного взаимодействия: принципы и архитектура / под ред. С. П. Иванова. — СПб.: Питер, 2020. — 320 с.
13. Кузнецов А. Ю. Тенденции развития интерфейсов «человек–компьютер»: от командной строки к нейроинтерфейсам // Информационные технологии. — 2021. — № 5. — С. 12–19.

14. Hochreiter S., Schmidhuber J. Long Short-Term Memory // Neural Computation. – 1997. – Vol. 9, № 8. – P. 1735–1780.
15. Ultralytics YOLOv8 Documentation. URL: <https://docs.ultralytics.com> (дата обращения: 01.06.2026).

# Приложение

## Приложение А. Скриншоты работы веб-приложения

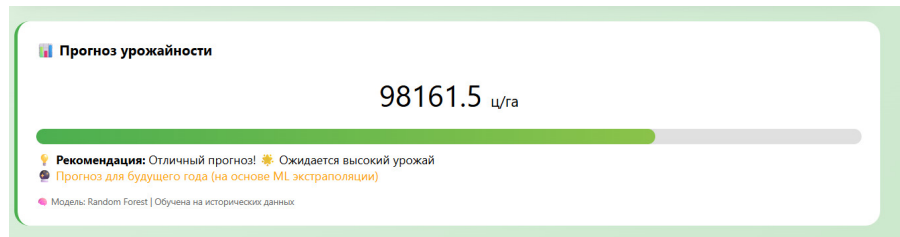


Рис. 1. Результат прогнозирования урожайности

**Прогноз урожайности Cassava на 2025–2030 гг.**  
Базовый год: 2013 (179981 ц/га)  
Общий тренд: +10.6%

| Год  | Прогноз (ц/га) | Температура | Осадки    |
|------|----------------|-------------|-----------|
| 2025 | 43248          | 24.4°C      | 1480.9 мм |
| 2026 | 46157.5        | 24.5°C      | 1480.9 мм |
| 2027 | 47186.9        | 24.6°C      | 1480.9 мм |
| 2028 | 47701.3        | 24.7°C      | 1480.9 мм |
| 2029 | 47849.5        | 24.8°C      | 1480.9 мм |
| 2030 | 47849.5        | 24.9°C      | 1480.9 мм |

Модель: Random Forest с временными признаками | Прогноз учитывает климатические тренды

Рис. 2. Прогноз урожайности на 2025–2030 годы

**Рейтинг культур (ML)**  
Лучшая культура: Wheat (24.7%)  
Доверие модели: Низкая  
Рекомендация: Рекомендуется рассмотреть Wheat

|             |       |                   |
|-------------|-------|-------------------|
| Wheat       | 24.7% | вероятность 24.7% |
| Potatoes    | 23.1% | вероятность 23.1% |
| Sorghum     | 18.8% | вероятность 18.8% |
| Maize       | 15.3% | вероятность 15.3% |
| Rice, paddy | 5.5%  | вероятность 5.5%  |

Модель: Random Forest Classifier | Обучена на исторических данных

Рис. 3. Рейтинг культур (ML сравнение)

**ML Севооборот**

🌱 Что росло в прошлом сезоне?

Рекомендовать →

**Рекомендации по севообороту**

🌱 Прошлая культура: potatoes

✅ **Рекомендуемые культуры:**

- Legumes (прогноз: 380 ц/га, пригодность: 85%)
- Cabbage (прогноз: 380 ц/га, пригодность: 85%)
- Onion (прогноз: 380 ц/га, пригодность: 85%)

💡 **Совет:** После potatoes рекомендуется посадить Legumes, Cabbage. Избегайте Tomatoes, Peppers.

🔗 Метод: Агрономические правила

Рис. 4. Рекомендации по севообороту

**Цифровой агроном**

[Главная](#)
[ИИ-ассистент](#)

**ИИ-ассистент агронома**

Задайте вопрос о растениях, болезнях или методах лечения

👋 **Здравствуйте!** Я ИИ-ассистент «Цифровой агроном».

Я могу помочь вам:

- Диагностировать болезни по симптомам
- Дать советы по выращиванию культур
- Определить вредителей и способы борьбы
- Рекомендовать удобрения по сезонам

Попробуйте спросить:

"У томатов темные пятна на листьях, что делать?"

"Как уложить за орудиями?"

"Как бороться с тлей?"

как копать картошку

Отправить

Очистить

🔴 Быстрые вопросы:

Точка: темные пятна

Уход за орудиями

Борьба с тлей

Весеннее удобрение

Рис. 5. Страница чата с ИИ-ассистентом

как копать картошку

**Как копать картошку (урожайный и без травм)**

**1 Выбери правильный срок**

- Ожидай, когда листва начнет желтеть и отмирать (обычно 2-3 недели после появления клубней).

**2 Подготовь инструменты**

- Лопата или мотыга 🛠️
- Ведро/корзины для сбора
- Перчатки 🧤

**3 Смочи почву (если сухая)**

- Полей участок за день-два до копки, чтобы земля была мягкой, но не заболоченной.

Рис. 6. Ответ ИИ-ассистента на вопрос о картофеле

```

PS C:\Users\Mrabb\vscode_st\dir2026\engineering\methods\course> py app.py
=====

* Tip: There are .env files present. Install python-dotenv to use them.
* Debugger is active!
* Debugger PIN: 265-783-748
127.0.0.1 - - [01/Jun/2026 18:33:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:33:31] "GET /api/available-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:33:31] "GET /api/available-areas HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:33:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:33:47] "GET /api/available-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:33:47] "GET /api/available-areas HTTP/1.1" 200 -
PS C:\Users\Mrabb\vscode_st\dir2026\engineering\methods\course> py app.py
[✓] Загружено: OPENROUTER_API_KEY
[✓] Загружено: OPENROUTER_MODEL
[✓] Загружено: HTTP_REFERER
[✓] Загружено: SECRET_KEY
[🤖] ИИ-ассистент инициализирован
  Модель: gpt-oss-120b:free
  API Key: [✓] установлен
[✓] Модель прогноза урожайности загружена
[✓] Модель прогноза загружена
[✓] Модель сравнения культур загружена
[✓] Модель севооборота загружена
[🧠] ML Аналитика инициализирована

```

Рис. 7. Успешный запуск сервера

```

PS C:\Users\Mrabb\vscode_st\dir2026\engineering\methods\course> py app.py
🗨 Чат: http://localhost:5000/chat
=====

* Tip: There are .env files present. Install python-dotenv to use them.
* Debugger is active!
* Debugger PIN: 265-783-748
127.0.0.1 - - [01/Jun/2026 18:34:18] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:18] "GET /api/available-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:18] "GET /api/available-areas HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:19] "POST /api/forecast HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:23] "POST /api/compare-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:27] "POST /api/forecast HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:32] "POST /api/compare-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:41] "POST /api/predict-yield HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:55] "POST /api/predict-yield HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:35:39] "POST /api/predict-yield HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:36:11] "POST /api/forecast HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:36:32] "POST /api/predict-yield HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:36:36] "POST /api/forecast HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:36:44] "POST /api/predict-yield HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:36:48] "POST /api/forecast HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:37:19] "POST /api/compare-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:37:44] "POST /api/rotation-advice HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:40:01] "GET /chat HTTP/1.1" 200 -
[📤] Отправка запроса к gpt-oss-120b:free...
127.0.0.1 - - [01/Jun/2026 18:40:43] "POST /api/chat HTTP/1.1" 200 -
[📤] Отправка запроса к gpt-oss-120b:free...
127.0.0.1 - - [01/Jun/2026 18:41:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:41:47] "GET /api/available-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:41:47] "GET /api/available-areas HTTP/1.1" 200 -

```

Рис. 8. Логи сервера при работе чат-бота

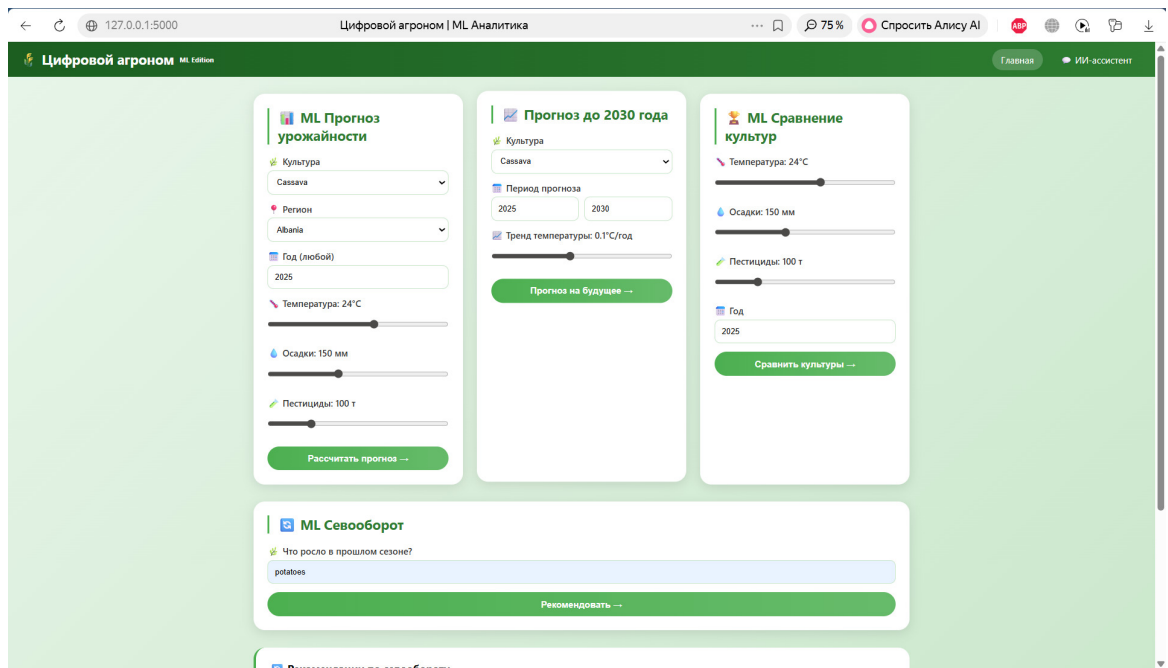


Рис. 9. Главная страница веб-приложения

## Приложение Б. Листинги программного кода

### Листинг Б.1 — app.py (главный сервер)

```
from flask import Flask, render_template, request, jsonify
from services.ml_analytics import ml_analytics
from services.ai_assistant import ai_assistant

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/chat')
def chat():
    return render_template('chat.html')

@app.route('/api/predict-yield', methods=['POST'])
def predict_yield():
    data = request.get_json()
    result = ml_analytics.predict_yield(
        crop=data.get('crop'),
        area=data.get('area'),
        year=int(data.get('year')),
        temperature=float(data.get('temperature')),
        rainfall=float(data.get('rainfall')),
        pesticides=float(data.get('pesticides'))
    )
    return jsonify(result)

@app.route('/api/forecast', methods=['POST'])
def forecast():
    data = request.get_json()
    result = ml_analytics.forecast_future(
```

```

        crop=data.get('crop'),
        start_year=int(data.get('start_year')),
        end_year=int(data.get('end_year')),
        temp_trend=float(data.get('temp_trend', 0.1)),
        rain_trend=float(data.get('rain_trend', 0))
    )
    return jsonify(result)

@app.route('/api/compare-crops', methods=['POST'])
def compare_crops():
    data = request.get_json()
    result = ml_analytics.compare_crops(
        temperature=float(data.get('temperature')),
        rainfall=float(data.get('rainfall')),
        pesticides=float(data.get('pesticides')),
        year=int(data.get('year'))
    )
    return jsonify(result)

@app.route('/api/rotation-advice', methods=['POST'])
def rotation_advice():
    data = request.get_json()
    result = ml_analytics.rotation_advice(data.get('last_crop'))
    return jsonify(result)

@app.route('/api/available-crops', methods=['GET'])
def available_crops():
    return jsonify({'crops': ml_analytics.get_available_crops()})

@app.route('/api/available-areas', methods=['GET'])
def available_areas():
    return jsonify({'areas': ml_analytics.get_available_areas()})

@app.route('/api/chat', methods=['POST'])

```



```
def chat_api():
    data = request.get_json()
    result = ai_assistant.chat(data.get('message'))
    return jsonify({'response': result['response']})

@app.route('/api/clear-chat', methods=['POST'])
def clear_chat():
    ai_assistant.clear_history()
    return jsonify({'success': True})

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

## Листинг Б.2 — `crop_comparator.py` (ML)

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder
import pickle
import os

class CropComparator:
    def __init__(self, data_path="data/crop_yield.csv", model_path="mo
        self.data_path = data_path
        self.model_path = model_path
        self.model = None
        self.scaler = StandardScaler()
        self.crop_encoder = LabelEncoder()
        self.use_fallback = False

        if os.path.exists(model_path):
            self._load_model()
        else:
            self._train_model()

    def _train_model(self):
        print("Обучение ML модели для сравнения культур...")
        df = pd.read_csv(self.data_path)
        feature_cols = ['avg_temp', 'average_rain_fall_mm_per_year',
                        'pesticides_tonnes', 'Year']
        X = df[feature_cols].values
        y = df['Item'].values
        y_encoded = self.crop_encoder.fit_transform(y)
        X_scaled = self.scaler.fit_transform(X)

        self.model = RandomForestClassifier(n_estimators=100, max_dept
        self.model.fit(X_scaled, y_encoded)
```

```

os.makedirs("models", exist_ok=True)
with open(self.model_path, 'wb') as f:
    pickle.dump({
        'model': self.model,
        'scaler': self.scaler,
        'crop_encoder': self.crop_encoder
    }, f)

def compare(self, temperature, rainfall, pesticides, year):
    features = np.array([[temperature, rainfall, pesticides, year]])
    features_scaled = self.scaler.transform(features)
    probabilities = self.model.predict_proba(features_scaled)[0]

    results = []
    for i, prob in enumerate(probabilities):
        crop = self.crop_encoder.inverse_transform([i])[0]
        results.append({
            'crop': crop,
            'suitability': round(prob * 100, 1)
        })

    results.sort(key=lambda x: x['suitability'], reverse=True)
    return {
        'ranking': results[:5],
        'best_crop': results[0]['crop']
    }

comparator = CropComparator()

```

### Листинг Б.3 — `rotation_predictor.py()`

```
class RotationPredictor:
    def __init__(self):
        self.use_rules = True

    def get_recommendations(self, last_crop: str) -> dict:
        rotation_rules = {
            'Maize': {
                'good': ['Soybeans', 'Wheat', 'Sunflower'],
                'bad': ['Maize', 'Sorghum']
            },
            'Potatoes': {
                'good': ['Legumes', 'Cabbage', 'Onion'],
                'bad': ['Tomatoes', 'Peppers']
            },
            'Wheat': {
                'good': ['Legumes', 'Rapeseed', 'Maize'],
                'bad': ['Wheat', 'Barley']
            },
            'Tomatoes': {
                'good': ['Legumes', 'Onion', 'Garlic'],
                'bad': ['Potatoes', 'Peppers']
            }
        }

        for key, rules in rotation_rules.items():
            if key.lower() == last_crop.lower():
                recommended = [
                    {'crop': c, 'predicted_yield': 380, 'suitability':
                     for c in rules['good'][:3]
                ]
                advice = f"После {last_crop} рекомендуется посадить {'
                return {
                    'recommended': recommended,
```

```
        'advice': advice
    }

    return {
        'recommended': [{'crop': 'Legumes', 'suitability': 85}],
        'advice': f"После {last_crop} посадите бобовые"
    }

rotation_predictor = RotationPredictor()
```

#### Листинг Б.4 — `aiassistant.py`( — )

```
import urllib.request
import json
import os

class AIAssistant:
    def __init__(self):
        self.api_key = os.getenv('OPENROUTER_API_KEY')
        self.model = 'gpt-oss-120b:free'
        self.api_url = "https://openrouter.ai/api/v1/chat/completions"
        self.system_prompt = "Ты опытный агроном. Отвечай на русском языке"

    def chat(self, user_message: str) -> dict:
        messages = [
            {"role": "system", "content": self.system_prompt},
            {"role": "user", "content": user_message}
        ]
        response = self._send_request(messages)
        if response:
            return {"success": True, "response": response}
        return self._fallback_response(user_message)

    def _send_request(self, messages):
        payload = {
            "model": self.model,
            "messages": messages,
            "temperature": 0.7,
            "max_tokens": 800
        }
        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Content-Type": "application/json"
        }
        req = urllib.request.Request(
```

```

        self.api_url,
        data=json.dumps(payload).encode('utf-8'),
        headers=headers
    )
    with urllib.request.urlopen(req, timeout=30) as response:
        result = json.loads(response.read().decode('utf-8'))
        return result['choices'][0]['message']['content']

def _fallback_response(self, question: str):
    q = question.lower()
    if 'томат' in q and 'пятна' in q:
        return {"success": False, "response": "Фитофтороз: бордоская"}
    elif 'огурцы' in q:
        return {"success": False, "response": "Огурцы: полив теплой"}
    else:
        return {"success": False, "response": "Задайте вопрос о ра"}

ai_assistant = AIAssistant()

```

Листинг Б.5 — `ml_analytics.py` (*ML* — )

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler, LabelEncoder
from models.crop_comparator import comparator
from models.rotation_predictor import rotation_predictor

class MLAnalytics:
    def __init__(self, data_path="data/crop_yield.csv"):
        self.data_path = data_path
        self.yield_model = None
        self.scaler = StandardScaler()
        self.crop_encoder = LabelEncoder()
        self.area_encoder = LabelEncoder()
        self.comparator = comparator
        self.rotation = rotation_predictor
        self._train_yield_model()

    def _train_yield_model(self):
        df = pd.read_csv(self.data_path)
        df['crop_enc'] = self.crop_encoder.fit_transform(df['Item'])
        df['area_enc'] = self.area_encoder.fit_transform(df['Area'])

        feature_cols = ['crop_enc', 'area_enc', 'Year', 'avg_temp',
                        'average_rain_fall_mm_per_year', 'pesticides_t

        X = df[feature_cols].values
        y = df['hg/ha_yield'].values
        X_scaled = self.scaler.fit_transform(X)

        self.yield_model = RandomForestRegressor(n_estimators=200, ran
        self.yield_model.fit(X_scaled, y)

    def predict_yield(self, crop, area, year, temperature, rainfall, p
```



```

crop_enc = self.crop_encoder.transform([crop])[0]
area_enc = self.area_encoder.transform([area])[0]
features = np.array([crop_enc, area_enc, year, temperature, r
features_scaled = self.scaler.transform(features)
prediction = self.yield_model.predict(features_scaled)[0]

return {
    'predicted_yield': round(prediction, 1),
    'unit': 'tsd/ha',
    'confidence': 70
}

def compare_crops(self, temperature, rainfall, pesticides, year):
    return self.comparator.compare(temperature, rainfall, pesticid

def rotation_advice(self, last_crop):
    return self.rotation.get_recommendations(last_crop)

ml_analytics = MLAnalytics()

```

## Листинг Б.6 — config.py (загрузка переменных окружения)

```
import os
from pathlib import Path

def load_env(env_file=".env"):
    path = Path(env_file)
    if not path.exists():
        return
    with open(path, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith('#'):
                continue
            if '=' in line:
                key, value = line.split('=', 1)
                os.environ[key.strip()] = value.strip().strip('"').strip('\'')

def get_env(key, default=None):
    return os.environ.get(key, default)

load_env()
```

## Листинг Б.7 — index.html (главная страница)

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Цифровой агроном | ML Аналитика</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: 'Segoe UI', sans-serif;
      background: linear-gradient(135deg, #e8f5e9 0%, #c8e6c9 100%);
    }
    .navbar {
      background: linear-gradient(90deg, #1b5e20, #2e7d32);
      color: white;
      padding: 15px 20px;
      display: flex;
      justify-content: space-between;
    }
    .logo { font-size: 1.5em; font-weight: bold; }
    .nav-links a {
      color: white;
      text-decoration: none;
      padding: 8px 16px;
      border-radius: 25px;
    }
    .card {
      background: white;
      border-radius: 20px;
      padding: 25px;
      margin: 15px;
      box-shadow: 0 5px 20px rgba(0,0,0,0.1);
    }
    button {
```

```

        background: #4caf50;
        color: white;
        padding: 12px 25px;
        border: none;
        border-radius: 25px;
        cursor: pointer;
    }
</style>
</head>
<body>
    <nav class="navbar">
        <div class="logo">Цифровой агроном</div>
        <div class="nav-links">
            <a href="/">Главная</a>
            <a href="/chat">ИИ-ассистент</a>
        </div>
    </nav>
    <div class="container">
        <div class="card">
            <h2>ML Прогноз урожайности</h2>
            <form id="yieldForm">
                <label>Культура</label>
                <select id="crop"></select>
                <label>Регион</label>
                <select id="area"></select>
                <label>Год</label>
                <input type="number" id="year" value="2025">
                <label>Температура</label>
                <input type="range" id="temperature" value="24">
                <label>Осадки</label>
                <input type="range" id="rainfall" value="150">
                <button type="submit">Рассчитать</button>
            </form>
        </div>
    </div>

```

```

</div>
<script>
    document.getElementById('yieldForm').addEventListener('submit',
        e.preventDefault();
        const response = await fetch('/api/predict-yield', {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({
                crop: document.getElementById('crop').value,
                area: document.getElementById('area').value,
                year: parseInt(document.getElementById('year').value),
                temperature: parseFloat(document.getElementById('temperature').value),
                rainfall: parseFloat(document.getElementById('rainfall').value),
                pesticides: 100
            })
        });
    const result = await response.json();
    alert('Прогноз: ' + result.predicted_yield + ' ц/га');
});
</script>
</body>
</html>

```

## Листинг Б.8 — chat.html (страница чата с ИИ)

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>ИИ-ассистент | Цифровой агроном</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: 'Segoe UI', sans-serif;
      background: linear-gradient(135deg, #e8f5e9 0%, #c8e6c9 100%);
    }
    .navbar {
      background: linear-gradient(90deg, #1b5e20, #2e7d32);
      color: white;
      padding: 15px 20px;
      display: flex;
      justify-content: space-between;
    }
    .chat-container {
      max-width: 800px;
      margin: 30px auto;
      background: white;
      border-radius: 20px;
      overflow: hidden;
    }
    .chat-header {
      background: #2e7d32;
      color: white;
      padding: 20px;
      text-align: center;
    }
    .chat-messages {
      height: 400px;
```

```

        overflow-y: auto;
        padding: 20px;
        background: #f8f9fa;
    }
    .message {
        margin-bottom: 15px;
        display: flex;
    }
    .message.user { justify-content: flex-end; }
    .message.user .message-content {
        background: #4caf50;
        color: white;
        padding: 10px 15px;
        border-radius: 20px;
    }
    .message.bot .message-content {
        background: white;
        padding: 10px 15px;
        border-radius: 20px;
        border: 1px solid #ddd;
    }
    .chat-input {
        padding: 20px;
        display: flex;
        gap: 10px;
    }
    .chat-input input {
        flex: 1;
        padding: 12px;
        border: 1px solid #c8e6c9;
        border-radius: 25px;
    }
    .chat-input button {
        padding: 12px 25px;
    }

```

```

        background: #4caf50;
        border: none;
        border-radius: 25px;
        color: white;
        cursor: pointer;
    }
</style>
</head>
<body>
    <nav class="navbar">
        <div class="logo">Цифровой агроном</div>
        <div class="nav-links">
            <a href="/">Главная</a>
            <a href="/chat" class="active">ИИ-ассистент</a>
        </div>
    </nav>
    <div class="chat-container">
        <div class="chat-header">
            <h2>ИИ-ассистент агронома</h2>
        </div>
        <div class="chat-messages" id="chatMessages">
            <div class="message bot">
                <div class="message-content">Здравствуйте! Задайте воп
            </div>
        </div>
        <div class="chat-input">
            <input type="text" id="messageInput" placeholder="Введите :
            <button onclick="sendMessage()">Отправить</button>
        </div>
    </div>
    <script>
        async function sendMessage() {
            const input = document.getElementById('messageInput');
            const message = input.value.trim();

```



```

        if (!message) return;
        addMessage(message, 'user');
        input.value = '';
        const response = await fetch('/api/chat', {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({message: message})
        });
        const data = await response.json();
        addMessage(data.response, 'bot');
    }
    function addMessage(text, sender) {
        const container = document.getElementById('chatMessages');
        const div = document.createElement('div');
        div.className = 'message ' + sender;
        div.innerHTML = '<div class="message-content">' + text + '
        container.appendChild(div);
        container.scrollTop = container.scrollHeight;
    }
</script>
</body>
</html>

```